

UNIT 1

PROBLEM SOLVING AND FUNDAMENTALS OF C

INTRODUCTION TO C

What is C?

C is a general-purpose and structured programming language used to develop different types of software and applications. It is known for its simplicity, speed and efficiency. C also provides features for working closely with memory and computer hardware, making it useful for both application and system programming.

Example

```
printf("Hello World");
```

This statement tells the computer to display Hello World on the screen.

History of C

C was developed by Dennis Ritchie at Bell Laboratories (Bell Labs), USA, in the early 1970s. It was mainly developed for system programming and became closely associated with the development of the UNIX operating system. C evolved from earlier languages such as BCPL and B.

Development of C

```
BCPL
  ↓
  B
  ↓
  C
```

Important Points

- Developer: Dennis Ritchie
- Organization: Bell Laboratories
- Developed: Early 1970s
- Earlier languages: BCPL and B

Dennis Ritchie

Dennis Ritchie was an American computer scientist who developed the C programming language at Bell Laboratories. He also played an important role in the development of the UNIX operating system along with Ken Thompson and other researchers. His work had a major influence on modern programming languages and software development.

Remember:

Dennis Ritchie → Developer of C language

Why Was C Developed?

C was developed to create system software and to make programming more efficient and easier than using low-level machine or assembly languages. It provides a combination of high-level programming features and low-level memory operations. This made C suitable for system programming and software that works closely with computer hardware.

Example

C was used in the development of parts of the UNIX operating system.

Features of C

C became popular because it provides simplicity, speed and efficient use of computer resources. It also supports structured programming and provides useful features such as functions, arrays, pointers and dynamic memory management.

Main Features

- Simple - C has a relatively small set of keywords and a simple syntax.
- Fast - C programs are generally fast because they are compiled into machine-level instructions.
- Portable - A C program can often be moved from one computer system to another with suitable modifications.
- Structured - A large program can be divided into smaller parts using functions and other structures.
- Efficient - C provides good performance and control over system resources.
- Middle-Level Language - C provides both high-level and low-level programming features.
- Supports Functions - Functions help divide a large program into smaller and manageable parts.
- Supports Pointers - Pointers allow programmers to work with memory addresses.

Applications of C

C is used in many areas of computer science and software development. Its speed, efficiency and ability to work closely with hardware make it especially useful for system-level programming.

Major Applications

- Operating Systems - Used to develop operating systems and system components.
- Embedded Systems - Used in controllers, machines and electronic devices.
- System Software - Used for software that works closely with computer hardware.
- Compilers - Used to develop compilers and language-processing tools.
- Database Systems - Used in the development of database software and components.
- Networking - Used to develop network-related software and protocols.
- Application Software - Can also be used to develop general-purpose applications.

Why Learn C?

C is important for computer science students because it teaches the basic concepts of programming clearly. It helps students understand program logic, variables, data types, operators, functions, arrays, pointers and memory. These concepts provide a strong foundation for learning other programming languages and computer science subjects.

Example

Understanding arrays, pointers and memory management in C can help students learn data structures and operating systems more easily.

PROBLEM SOLVING

What is a Problem?

A problem is a task or situation for which we need to find a solution. In programming, a problem is given to the programmer, and the programmer develops a logical solution that can be executed by a computer.

Examples

- Finding the sum of two numbers
- Calculating the area of a circle
- Finding the largest number
- Checking whether a number is even or odd

What is Problem Solving?

Problem solving is the process of understanding a problem and finding a proper way to solve it. In programming, we first understand the problem, identify the required input and output, develop the logic and then write the program.

Example

To find the sum of two numbers:

```
Input → Two numbers
Processing → Addition
Output → Sum
```

Input

Input is the data given to the computer for processing. It can be a number, character, name or any other information required by the program. The program uses this input to perform the required operation.

Example

```
A = 10
B = 20
```

Here, 10 and 20 are the input values.

Processing

Processing means performing an operation on the input data to get the required result. It may include calculations, comparisons or other operations depending on the problem.

Example

```
Sum = A + B
     = 10 + 20
     = 30
```

Here, addition is the processing.

Output

Output is the result produced by the computer after processing the input. It is the information that we want to obtain from the program.

Example

```
Input:      10, 20
Processing: 10 + 20
Output:     30
```

Here, 30 is the output.

Logic

Logic is the way of thinking used to solve a problem. It tells us what operation should be performed and in what order to get the correct result. Good logic is important because a program can have correct syntax but still produce an incorrect result if the logic is wrong.

Example

To check whether a number is even or odd:

```
If number % 2 == 0
    → Even
Otherwise
    → Odd
```

Basic Problem-Solving Steps

A programmer follows a sequence of steps to convert a problem into a working program. These steps help to understand the problem, develop the solution, write the program and verify that the result is correct.

Steps

- Understand the Problem – Find out what the problem is asking and what result is required.
- Identify Input and Output – Decide what data is needed and what result should be produced.
- Develop the Logic – Decide how the input should be processed.
- Prepare the Algorithm – Write the solution as step-by-step instructions.
- Prepare the Flowchart – Represent the solution using standard flowchart symbols.
- Write the Program – Convert the solution into C language.
- Test and Debug – Run the program, check the result and correct errors.

ALGORITHMS AND FLOWCHARTS

Algorithm

An algorithm is a step-by-step procedure used to solve a particular problem. It tells us what should be done first, what should be done next and how to obtain the final result. An algorithm is prepared before writing the actual program so that the logic becomes clear.

Example - Adding Two Numbers

```
Step 1: Start
Step 2: Read A and B
Step 3: Calculate Sum = A + B
Step 4: Display Sum
Step 5: Stop
```

Characteristics of an Algorithm

A good algorithm should be clear, definite and finite. It should accept the required input, produce the required output and contain only useful steps for solving the problem.

Characteristics

- Clear – Each step should be simple and easy to understand.
- Definite – Every step should have only one clear meaning.
- Finite – The algorithm should have a limited number of steps and must eventually stop.

- Input - It should accept the required data.
- Output - It should produce the required result.
- Effective - Every step should be useful in solving the problem.

Advantages of an Algorithm

An algorithm helps the programmer understand and plan the solution before writing the actual program. It makes the logic easier to check and also helps in converting the solution into different programming languages.

Advantages

- Easy to Understand - The solution is written as simple step-by-step instructions.
- Helps in Program Development - It provides a clear plan for writing the program.
- Helps Find Errors - Logical mistakes can be identified before coding.
- Language Independent - The same algorithm can be implemented in C, C++, Java or Python.
- Easy to Modify - Steps can be changed when the requirements change.
- Useful for Documentation - It can be used to record and explain the logic of a program.

Flowchart

A flowchart is a graphical representation of an algorithm. It uses standard symbols to represent different steps of a solution and arrows to show the direction of the process. Flowcharts make the logic easier to understand because the solution is represented visually.

Example

```
graph TD; START --> ReadA[Read A, B]; ReadA --> Sum[Sum = A + B]; Sum --> DisplaySum[Display Sum]; DisplaySum --> STOP;
```

Basic Flowchart Symbols

Terminator - Oval

The oval symbol is used to show the beginning and end of a flowchart. It is normally used for START and STOP.

Process - Rectangle

The rectangle represents a calculation or processing operation that the computer needs to perform.

Example:

```
Sum = A + B
```

Input/Output - Parallelogram

The parallelogram is used when data is entered into the computer or when the result is displayed.

Examples:

```
Read A
Display Sum
```

Decision - Diamond

The diamond is used when the program needs to check a condition and choose between two or more paths.

Example:

```
Is A > B?
```

The answer may be Yes or No.

Flow Line - Arrow

The arrow shows the direction in which the flowchart moves from one step to another.

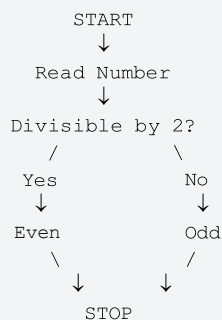
Example of a Flowchart - Even or Odd

A number is even when it is completely divisible by 2. If the number is not divisible by 2, it is considered odd.

Algorithm

```
Step 1: Start
Step 2: Read Number
Step 3: Check whether Number is divisible by 2
Step 4: If yes, display Even
Step 5: Otherwise, display Odd
Step 6: Stop
```

Flow



PROGRAMMING LANGUAGES

What is a Programming Language?

A programming language is a language used by programmers to give instructions to a computer. Since computers cannot directly understand normal human languages, programming languages provide a way for humans to communicate instructions to the computer.

Examples

```
C
C++
Java
Python
JavaScript
```

Why Do We Need Programming Languages?

A computer works using machine language, which consists of 0 and 1. It is very difficult for humans to write large programs directly in binary. Programming languages make it easier for programmers to write instructions and develop software.

Example

Instead of writing binary instructions, a programmer can write:

```
printf("Hello");
```

Types of Programming Languages

Programming languages are commonly classified into low-level languages and high-level languages. Low-level languages are closer to computer hardware, while high-level languages are designed to be easier for humans to understand and use.

Main Types

Low-Level Languages

- Machine Language
- Assembly Language

High-Level Languages

- C
- C++
- Java
- Python
- JavaScript

Machine Language

Machine language is the lowest-level language understood directly by the computer. It uses only binary digits, 0 and 1, to represent instructions. It is very difficult for humans to write and understand because the instructions are represented in binary form.

Example

```
10110000 01100001
```

Assembly Language

Assembly language uses short words or symbols called mnemonics instead of writing instructions only in binary. It is easier to understand than machine language but is still closely related to computer hardware.

Example

```
MOV A, B
ADD A, C
```

An assembler is used to convert assembly language into machine language.

High-Level Language

A high-level language is designed to be easier for humans to read, write and understand. It uses words and syntax that are closer to human language than machine language. Examples include C, C++, Java, Python and JavaScript.

Example

```
sum = a + b;
```

This is much easier for a programmer to understand than binary instructions.

C as a Programming Language

C is a general-purpose, structured programming language developed by Dennis Ritchie at Bell Laboratories in the early 1970s. It is widely used for system software as well as general programming. C provides features for both high-level programming and low-level operations.

Example

```
printf("Hello World");
```

Why is C Called a Middle-Level Language?

C is often called a middle-level language because it provides features of both high-level and low-level languages. It provides high-level features such as functions, loops and decision-making, while also providing low-level features such as pointers and direct memory manipulation.

Example

```
int a = 10;  
int b = 20;  
int sum = a + b;
```

C can also work directly with memory using pointers, which is one reason it is considered a middle-level language.

COMPILER AND INTERPRETER

Compiler

A compiler is software that translates a program written in a high-level language into machine-level code so that the computer can execute it. The compiler generally translates the program before execution and also checks the program for errors. C programs are generally compiled before execution.

Example

```
C Program  
↓  
Compiler  
↓  
Machine Code  
↓  
Output
```

Interpreter

An interpreter is software that translates and executes a program one statement at a time. It reads a statement, translates it, executes it and then moves to the next statement. If an error is found, execution generally stops at that point.

Example

```
Statement 1 → Translate → Execute
Statement 2 → Translate → Execute
Statement 3 → Translate → Execute
```

Compiler vs Interpreter

Compiler	Interpreter
Translates the program before execution	Translates and executes step by step
Generally reports errors during compilation	Generally reports errors as execution reaches them
Produces object/executable code	Usually executes through the interpreter
Execution is generally faster after compilation	Execution is generally slower
C uses a compiler	Python commonly uses an interpreter

Compiler in C

C is a compiled programming language. The C source program is processed by a compiler, which converts it into object or machine code and then into an executable program. The executable can then be run to produce the required output.

Process

```
C Source Program
↓
Compiler
↓
Object / Machine Code
↓
Executable
↓
Output
```

Why Do We Need a Compiler?

A computer cannot directly understand C statements such as `printf()`, `if` or `while`. The compiler converts these C instructions into machine-level instructions that the computer can understand. It also helps identify syntax errors before the program is executed.

Example

```
printf("Hello")
```

Here, the semicolon is missing. The compiler reports an error so that the programmer can correct it.

STRUCTURE OF A C PROGRAM

What is a C Program?

A C program is a set of instructions written in C language to perform a particular task. A C program generally contains elements such as header files, the `main()` function, declarations, statements and a

return statement.

Simple C Program

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

Basic Structure of a C Program

A beginner-level C program can generally be understood through the following parts. Not every program must contain all the parts separately, but this structure is useful for learning the basics.

```
Comments
↓
Header Files
↓
main() Function
↓
Declarations
↓
Statements
↓
return Statement
```

Comments

Comments are notes written inside a program to explain the code. They are ignored by the compiler and do not affect program execution. Comments make the program easier to read, understand and maintain.

Single-Line Comment

```
// This is a single-line comment
```

Multi-Line Comment

```
/*
   This is a
   multi-line comment
*/
```

Header Files

A header file contains declarations and information about functions that can be used in a C program. Header files are included using the `#include` directive. For input and output operations, we commonly use `stdio.h`.

Example

```
#include <stdio.h>
```

`stdio.h` provides functions such as `printf()` and `scanf()`.

main() Function

The `main()` function is the starting point of execution of a C program. When a C program is run, execution begins from the `main()` function. The statements that need to be executed are written inside its braces.

Example

```
int main()
{
    printf("Hello");
}
```

Braces { }

Curly braces are used to show the beginning and end of a block of statements. The opening brace { starts the block, while the closing brace } ends it.

Example

```
int main()
{
    printf("Hello");
}
```

Everything between { and } belongs to the `main()` function.

Statements

A statement is an instruction given to the computer to perform an operation. Most C statements end with a semicolon ;. Different statements can be used for input, output, calculations, decisions and other operations.

Example

```
printf("Hello World");
```

This statement displays Hello World on the screen.

return Statement

The `return` statement is used to return a value from a function. In the `main()` function, `return 0;` commonly indicates that the program has completed successfully.

Example

```
int main()
{
    printf("Hello");
    return 0;
}
```

Complete Simple C Program

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

Explanation

- `#include <stdio.h>` → Includes standard input/output functions.
- `int main()` → Defines the main function where execution begins.
- `printf("Hello World");` → Displays the message.

- `return 0;` → Indicates successful completion.

Important Points

- Every C program normally has a `main()` function.
- Most C statements end with a semicolon `;`.
- `{ }` define a block of statements.
- `printf()` is used to display output.
- `#include` is used to include header files.
- C is case-sensitive.

CHARACTER SET AND TOKENS

Character Set

A character set is a collection of characters that can be used in a C program. These characters are combined to form keywords, identifiers, constants, operators and other parts of the program.

Main Categories

Letters

```
A B C ... Z  
a b c ... z
```

Both uppercase and lowercase letters can be used.

Digits

```
0 1 2 3 4 5 6 7 8 9
```

Digits are used in numbers, constants and identifiers.

Special Characters

Some commonly used special characters are:

```
+ - * / % = < >  
( ) { } [ ]  
; , # &
```

White Spaces

White spaces include:

- Space
- Tab
- New line

They mainly improve the readability of the program.

Tokens

A token is the smallest individual unit of a C program that has a meaningful purpose. A C program is made up of different types of tokens, just as a sentence is made up of words and symbols.

Example

```
int age = 20;
```

Here:

- `int` → Keyword
- `age` → Identifier

- `=` → Operator
- `20` → Constant
- `;` → Special symbol

Types of Tokens

- Keywords
- Identifiers
- Constants
- String literals
- Operators
- Special symbols / Punctuators

KEYWORDS, IDENTIFIERS, CONSTANTS AND VARIABLES

Keywords

Keywords are reserved words in C that have a predefined meaning. They are provided by the C language and cannot normally be used as names of variables, functions or other user-defined elements.

Example

```
int age;
```

Here, `int` is a keyword because it tells the compiler that `age` is an integer variable.

Traditional 32 Keywords in C

Keyword	Keyword	Keyword	Keyword
<code>auto</code>	<code>break</code>	<code>case</code>	<code>char</code>
<code>const</code>	<code>continue</code>	<code>default</code>	<code>do</code>
<code>double</code>	<code>else</code>	<code>enum</code>	<code>extern</code>
<code>float</code>	<code>for</code>	<code>goto</code>	<code>if</code>
<code>int</code>	<code>long</code>	<code>register</code>	<code>return</code>
<code>short</code>	<code>signed</code>	<code>sizeof</code>	<code>static</code>
<code>struct</code>	<code>switch</code>	<code>typedef</code>	<code>union</code>
<code>unsigned</code>	<code>void</code>	<code>volatile</code>	<code>while</code>

Important Keywords and Their Uses

- `int` - Used to declare integer variables.
- `float` - Used to declare floating-point variables.
- `char` - Used to store a single character.
- `if` - Used to check a condition.
- `else` - Used when the `if` condition is false.
- `for` - Used to repeat statements.
- `while` - Repeats statements while a condition is true.

- `do` - Used with `while` for loop execution.
- `break` - Stops a loop or switch.
- `continue` - Skips the current loop iteration.
- `switch` - Selects one option from multiple choices.
- `case` - Defines an option inside `switch`.
- `default` - Executes when no case matches.
- `return` - Returns a value from a function.
- `void` - Represents no value.
- `const` - Used when a value should not be modified.
- `sizeof` - Used to find the size occupied by a type or object.

Important Points About Keywords

- Keywords are reserved words in C.
- They have a predefined meaning.
- They cannot normally be used as variable or function names.
- Keywords are generally written in lowercase.
- The traditional C language has 32 commonly taught keywords.

Example

```
int int;
```

This is invalid because `int` is already a keyword.

IDENTIFIERS

Identifiers

An identifier is a name given by the programmer to elements such as variables, functions, arrays, structures and other user-defined elements. Identifiers help us refer to these elements in a program.

Example

```
int age;  
float salary;
```

Here, `age` and `salary` are identifiers.

Rules for Naming Identifiers

Must Begin With a Letter or Underscore

An identifier can start with an alphabet or underscore, but it cannot start with a number.

<code>age</code>	✓
<code>_marks</code>	✓
<code>2marks</code>	✗

Can Contain Letters, Digits and Underscore

After the first character, letters, digits and underscores can be used.

```
student1  
total_marks  
marks2026
```

No Spaces

Spaces are not allowed inside identifiers.

```
student name    X
student_name    ✓
```

Keywords Cannot Be Used

A keyword cannot be used as an identifier.

```
int      X
float    X
while    X
```

Special Characters Are Not Allowed

Characters such as @, #, \$, % and & cannot normally be used in identifiers.

```
student@name    X
student_name    ✓
```

C Is Case-Sensitive

C treats uppercase and lowercase letters as different.

```
int marks;
int Marks;
```

Here, marks and Marks are two different identifiers.

Valid Identifiers

```
age
studentName
total_marks
marks1
_num
averageMarks
```

Invalid Identifiers

```
2marks
student name
total-marks
float
student@name
```

Good Naming of Identifiers

Identifiers should be meaningful and easy to understand. A meaningful name tells us what the variable or function is used for and makes the program easier to read.

Example

```
int studentAge;
float averageMarks;
```

These names are easier to understand than:

```
int x;
float a;
```

CONSTANTS

Constants

A constant is a value that does not change during the execution of a program. Constants may be numbers, characters or strings that represent fixed values used in a program.

Types of Constants

Integer Constant

A whole number without a decimal point.

```
10, 25, -5, 100
```

Real / Floating-Point Constant

A number containing a decimal point.

```
10.5, 25.75, -3.14
```

Character Constant

A single character enclosed in single quotation marks.

```
'A'  
'5'  
'#'
```

String Constant

A group of characters enclosed in double quotation marks.

```
"Hello"  
"Welcome to C"
```

VARIABLES

Variables

A variable is a named memory location used to store data. The value stored in a variable can change during program execution, which is why it is called a variable.

Example

```
int age = 20;  
  
age = 21;
```

Here, `age` first stores 20 and later stores 21.

Difference Between Constants and Variables

Constant	Variable
Value normally does not change	Value can change
Represents fixed data	Stores changing data
Example: 100	Example: marks
Used when a fixed value is required	Used when data may change

Example

```
int marks = 80;
```

Here, 80 is a constant value and `marks` is a variable.

Why Do We Need Variables?

Programs often need to store and work with values while they are running. Instead of repeatedly using actual values, we give them meaningful names and use those names in calculations and other operations.

Example

```
int length = 10;
int breadth = 5;
int area = length * breadth;
```

Here, `length`, `breadth` and `area` are variables.

Constant Variable Using `const`

C allows us to declare a variable whose value should not be changed by using the `const` keyword.

Example

```
const float PI = 3.14;
```

Here, `PI` is intended to remain unchanged during the program.

DATA TYPES AND VARIABLE DECLARATION

Data Type

A data type tells the C compiler what type of data a variable can store. It also helps the compiler determine how the data should be stored in memory.

Example

```
int age = 20;
float salary = 25000.50;
char grade = 'A';
```

Here, `int`, `float` and `char` are data types.

Basic Data Types in C

Data Type	Used For	Example
<code>int</code>	Whole numbers	<code>int age = 20;</code>
<code>float</code>	Decimal numbers	<code>float avg = 75.5;</code>
<code>char</code>	Single character	<code>char grade = 'A';</code>
<code>double</code>	More precise decimal values	<code>double x = 12.3456;</code>
<code>void</code>	No value	<code>void display()</code>

`int`

The `int` data type is used to store whole numbers without decimal values. It can store positive and negative integer values.

```
int age = 20;
int marks = 85;
```

float

The `float` data type is used to store decimal numbers. It is suitable when normal floating-point precision is sufficient.

```
float height = 5.8;
float average = 75.5;
```

char

The `char` data type is used to store a single character. The character is written inside single quotation marks.

```
char grade = 'A';
char gender = 'M';
```

double

The `double` data type is used to store decimal numbers with greater precision than `float`.

```
double distance = 12345.6789;
```

void

The `void` type represents no value or no specific data type. It is commonly used for functions that do not return a value.

```
void display()
{
    printf("Hello");
}
```

Type Modifiers

Type modifiers are keywords used with basic data types to modify their size, range or nature. The commonly used modifiers are `short`, `long`, `signed` and `unsigned`.

short

Used to declare a smaller integer type where supported.

```
short int age;
```

long

Used when a larger range of integer values is required.

```
long int population;
```

signed

Allows positive and negative values, along with zero.

```
signed int temperature;
```

unsigned

Allows zero and positive values. Since negative values are not stored, it can provide a larger positive range.

```
unsigned int marks;
```

Variable Declaration

Variable declaration means telling the compiler about a variable before using it. During declaration, we specify the data type and the name of the variable.

Syntax

```
data_type variable_name;
```

Example

```
int age;  
float salary;  
char grade;
```

Variable Initialization

Initialization means giving a variable its first value. A variable can be initialized at the time of declaration by using the assignment operator =.

Example

```
int age = 20;
```

Here, age is declared as an integer and its initial value is 20.

Declaration and Initialization Together

A variable can be declared and initialized in the same statement.

```
int marks = 85;  
float average = 75.5;  
char grade = 'A';
```

Declaration First, Initialization Later

A variable can also be declared first and assigned a value later.

```
int age;  
  
age = 20;
```

Multiple Variable Declaration

C allows multiple variables of the same data type to be declared in one statement.

```
int a, b, c;
```

They can also be initialized together:

```
int a = 10, b = 20, c = 30;
```

Changing the Value of a Variable

The value stored in a variable can be changed during program execution. This ability to change the stored value is the main reason it is called a variable.

```
int marks = 50;

marks = 80;
```

Initially `marks` is 50, and later its value becomes 80.

Declaration vs Initialization

Declaration	Initialization
Tells the compiler the type and name	Gives the variable its first value
Example: <code>int age;</code>	Example: <code>int age = 20;</code>

OPERATORS AND EXPRESSIONS

What is an Operator?

An operator is a symbol used to perform an operation on one or more values or variables. Operators are used for calculations, comparisons, assigning values and making logical decisions.

Example

```
sum = a + b;
```

Here, `+` performs addition and `=` assigns the result to `sum`.

What is an Operand?

An operand is the value or variable on which an operator performs an operation.

Example

```
a + b
```

Here, `a` and `b` are operands, while `+` is the operator.

Types of Operators in C

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment and decrement operators
- Conditional operator
- Bitwise operators
- Special operators

Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication and division.

Operator	Meaning	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Remainder	$a \% b$

Example

```
int a = 10, b = 3;

a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
```

The % operator gives the remainder after division.

Relational Operators

Relational operators are used to compare two values. The result is generally treated as true (1) or false (0) in C.

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

Example

```
10 > 5
```

Result:

```
1 → True
```

Logical Operators

Logical operators are used to combine or reverse conditions. They are commonly used in decision-making statements.

Operator	Meaning
&&	AND
	OR
!	NOT

Example

```
age >= 18 && age <= 60
```

Here, both conditions must be true for the complete expression to be true.

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Meaning	Example
=	Assignment	a = 10
+=	Add and assign	a += 5
-=	Subtract and assign	a -= 5
*=	Multiply and assign	a *= 5
/=	Divide and assign	a /= 5
%=	Remainder and assign	a %= 5

Example

```
a += 5;
```

This is equivalent to:

```
a = a + 5;
```

Increment Operator

The increment operator ++ increases the value of a variable by 1.

```
int a = 5;
a++;
```

After execution:

```
a = 6
```

Decrement Operator

The decrement operator -- decreases the value of a variable by 1.

```
int a = 5;
a--;
```

After execution:

```
a = 4
```

Conditional Operator

The conditional operator ?: is also called the ternary operator because it uses three parts. It is useful when we want to make a simple decision in a single expression.

Example

```
result = (a > b) ? a : b;
```

If `a > b` is true, `a` is assigned to `result`; otherwise, `b` is assigned.

Bitwise Operators

Bitwise operators perform operations on the individual bits of integer values. They are mainly useful in low-level programming and situations where direct bit manipulation is required.

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

Example

```
a & b
```

This performs a bitwise AND operation on `a` and `b`.

Expression

An expression is a combination of constants, variables and operators that produces a value. Expressions are used to perform calculations and other operations in a program.

Example

```
a + b
```

Here, `a` and `b` are operands and `+` is the operator.

Another Example

```
total = marks1 + marks2;
```

The expression `marks1 + marks2` produces a value that is assigned to `total`.

OPERATOR PRECEDENCE AND TYPE CONVERSION

Operator Precedence

Operator precedence tells us which operator should be performed first when an expression contains more than one operator. Operators with higher precedence are generally evaluated before operators with lower precedence.

Example

```
result = 10 + 5 * 2;
```

First:

```
5 * 2 = 10
```

Then:

```
10 + 10 = 20
```

Therefore:

```
result = 20
```

Basic Order of Precedence

Priority	Operators	Meaning
High	()	Parentheses
	* / %	Multiplication, Division, Remainder
	+ -	Addition, Subtraction
	< > <= >=	Relational
	== !=	Equality
	&&	Logical AND
Low	=	Assignment

This is a simplified beginner-level order based on the material.

Parentheses

Parentheses can be used to change the normal order of evaluation. The expression inside the parentheses is evaluated first.

Example

```
result = (10 + 5) * 2;
```

First:

```
10 + 5 = 15
```

Then:

```
15 * 2 = 30
```

Therefore:

```
result = 30
```

Associativity

When two or more operators have the same precedence, associativity determines the direction in which they are evaluated. Many arithmetic operators such as +, -, *, and / are evaluated from left to right when they have the same precedence.

Example

```
result = 20 / 5 * 2;
```

Evaluation:

```
20 / 5 = 4  
4 * 2 = 8
```

Therefore:

```
result = 8
```

Type Conversion

Type conversion means changing a value from one data type to another. It is useful when different data types are used together or when a particular type of result is required.

Example

```
10 → 10.0
```

There are two main types:

- Implicit Type Conversion
- Explicit Type Conversion

Implicit Type Conversion

Implicit type conversion happens automatically by the compiler. The programmer does not need to write any special instruction for the conversion.

Example

```
int a = 10;  
float b;  
  
b = a;
```

Here, the integer value 10 is automatically converted to a floating-point value.

```
int → float
```

Explicit Type Conversion

Explicit type conversion is performed by the programmer using a type cast. It is used when we want to convert a value into a specific data type.

Syntax

```
(data_type) value
```

Example

```
int a = 5, b = 2;  
  
float result = (float)a / b;
```

Here, a is temporarily converted from int to float.

Therefore:

```
5 / 2 = 2.5
```

Integer Division

When two integers are divided, C performs integer division, and the decimal part is discarded.

Example

```
int a = 5;
int b = 2;

printf("%d", a / b);
```

Output:

```
2
```

The mathematical result is 2.5, but because both operands are integers, C gives 2.

Using Type Casting

To obtain the decimal result, we can use type casting.

```
int a = 5;
int b = 2;

float result = (float)a / b;

printf("%f", result);
```

Output:

```
2.500000
```

Here, (float) a converts a to a floating-point value before division.

Remember:

- Implicit → Automatic conversion
- Explicit → Programmer-controlled conversion

STANDARD INPUT AND OUTPUT

Standard Input and Output Functions

Input functions are used to receive data from the user, while output functions are used to display information on the screen. Most standard input and output functions are available through the `stdio.h` header file.

```
#include <stdio.h>
```

printf() Function

The `printf()` function is used to display output on the screen. It can display text, numbers, characters and values stored in variables.

Example

```
printf("Hello World");
```

Output:

```
Hello World
```

With a Variable

```
int age = 20;

printf("%d", age);
```

Output:

scanf() Function

The `scanf()` function is used to take input from the user through the keyboard. It stores the entered value in a variable, and for most ordinary variables we use `&` to provide the variable's address.

Example

```
int age;  
  
scanf("%d", &age);
```

If the user enters:

20

the value 20 is stored in `age`.

getchar() Function

The `getchar()` function is used to read one character from the keyboard. It is useful when the program needs to accept a single character as input.

Example

```
char ch;  
  
ch = getchar();
```

If the user enters A, the character A is stored in `ch`.

putchar() Function

The `putchar()` function is used to display one character on the screen. It accepts a character and prints it as output.

Example

```
char ch = 'A';  
  
putchar(ch);
```

Output:

A

puts() Function

The `puts()` function is used to display a string or group of characters. It automatically moves the cursor to the next line after displaying the string.

Example

```
puts("Welcome to C Programming");
```

Output:

Welcome to C Programming

gets() and fgets()

gets() was traditionally used to read a string from the keyboard, but it is unsafe because it does not limit the amount of input. It was removed from the C standard in C11. Modern C programs should use fgets() for safe line input.

Example

```
char name[30];

fgets(name, sizeof(name), stdin);
```

This reads a line of text within the specified size.

Common Standard Input and Output Functions

Function	Purpose
printf()	Displays formatted output
scanf()	Takes formatted input
getchar()	Reads one character
putchar()	Displays one character
puts()	Displays a string
fgets()	Reads a line/string safely

Simple C Program Using printf() and scanf()

```
#include <stdio.h>

int main()
{
    int age;

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Your age is %d", age);

    return 0;
}
```

Sample Input

```
Enter your age: 20
```

Output

```
Your age is 20
```

Remember:

printf() is mainly used for output, while scanf() is mainly used for input.

FORMAT SPECIFIERS AND ESCAPE SEQUENCES

Format Specifiers

Format specifiers are special symbols used with functions such as `printf()` and `scanf()` to tell C what type of data is being displayed or entered. They generally begin with the `%` symbol.

Example

```
int age = 20;

printf("%d", age);
```

Here `%d` tells C that age is an integer.

Common Format Specifiers

Format Specifier	Data Type	Example
<code>%d</code>	Integer	<code>int age = 20;</code>
<code>%f</code>	Float	<code>float price = 25.5;</code>
<code>%lf</code>	Double in <code>scanf()</code>	<code>double x = 12.5;</code>
<code>%c</code>	Character	<code>char grade = 'A';</code>
<code>%s</code>	String	<code>char name[] = "Ravi";</code>
<code>%u</code>	Unsigned integer	<code>unsigned int n = 10;</code>
<code>%ld</code>	Long integer	<code>long int n = 10000;</code>

`%d`

`%d` is used for displaying or reading an integer value.

```
int age = 20;

printf("%d", age);
```

Output:

```
20
```

`%f`

`%f` is commonly used with float values.

```
float marks = 85.5;

printf("%f", marks);
```

Output:

```
85.500000
```

`%c`

`%c` is used for a single character.

```
char grade = 'A';

printf("%c", grade);
```

Output:

A

%s

%s is used to display or read a string, which is a sequence of characters.

```
char name[] = "Vamshi";

printf("%s", name);
```

Output:

```
Vamshi
```

Why Are Format Specifiers Required?

C needs to know what type of data is being processed by functions such as `printf()` and `scanf()`. Format specifiers provide this information and tell the function how the value should be interpreted.

Example

```
int age = 20;

printf("%d", age);
```

Here %d tells `printf()` to treat age as an integer.

Escape Sequences

An escape sequence is a special combination of characters used to represent a special action or character in a C program. It usually begins with a backslash \ and is mainly used inside strings and character constants.

Example

```
printf("Hello\nWorld");
```

Output:

```
Hello
World
```

Here \n moves the cursor to the next line.

Common Escape Sequences

Escape Sequence	Meaning
\n	New line
\t	Horizontal tab
\b	Backspace
\r	Carriage return
\a	Alert / beep
\\	Backslash
\'	Single quotation mark

Escape Sequence	Meaning
\"	Double quotation mark
\0	Null character

\n - New Line

\n moves the output to the next line.

```
printf("Hello\nWorld");
```

Output:

```
Hello
World
```

\t - Tab

\t inserts a horizontal tab space.

```
printf("Name\tMarks");
```

Output:

```
Name      Marks
```

\\ - Backslash

\\ is used when we want to display a backslash character inside a string.

```
printf("C:\\Program");
```

Output:

```
C:\Program
```

\ " - Double Quote

\ " is used to display double quotation marks inside a string.

```
printf("He said \"Hello\"");
```

Output:

```
He said "Hello"
```

\ ' - Single Quote

\ ' is used to represent a single quotation mark.

```
printf("\'");
```

Output:

```
'
```

Format Specifier vs Escape Sequence

Format Specifier	Escape Sequence
Tells C what type of data is being handled	Tells C to perform a special action
Begins with %	Begins with \
Example: %d	Example: \n
Used with printf() and scanf()	Mainly used inside strings/character constants

Example

```
printf("Age = %d\n", age);
```

Here:

- %d → Format specifier
- \n → Escape sequence

WRITING AND TRACING SIMPLE C PROGRAMS

Writing a Simple C Program

Writing a C program means converting a solution or logic into C language instructions. A simple program normally contains a header file, main() function, variables, statements and a return statement.

Example - Addition of Two Numbers

```
#include <stdio.h>

int main()
{
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;

    printf("Sum = %d", sum);

    return 0;
}
```

Understanding the Program

```
#include <stdio.h>
```

This includes the standard input/output header file. It is required for functions such as printf() and scanf().

```
int main()
```

The main() function is the starting point of program execution.

```
int a, b, sum;
```

These are integer variables used to store the two input values and their sum.

```
scanf("%d %d", &a, &b);
```

This takes two integer values from the user and stores them in a and b.

```
sum = a + b;
```

This performs addition and stores the result in sum.

```
printf("Sum = %d", sum);
```

This displays the calculated sum.

```
return 0;
```

This indicates that the program has completed successfully.

What is Tracing a Program?

Tracing means following a program step by step to understand how the values of variables change and what output is produced. It is especially useful for beginners because it helps them understand the flow and logic of a program without actually running it.

Example

```
int a = 10;
int b = 5;
int sum;

sum = a + b;
```

Trace

Step	Statement	a	b	sum
1	a = 10	10	–	–
2	b = 5	10	5	–
3	sum = a + b	10	5	15

Final Value

```
sum = 15
```

Example - Find the Area of a Rectangle

```
#include <stdio.h>

int main()
{
    int length = 10;
    int breadth = 5;
    int area;

    area = length * breadth;

    printf("Area = %d", area);

    return 0;
}
```

Tracing

```
length = 10
breadth = 5

area = length * breadth
      = 10 * 5
      = 50
```

Output

```
Area = 50
```

UNIT 1 – IMPORTANT QUESTIONS

2 Marks

1. Who developed C language?
2. Define a variable.
3. What is a keyword? Give examples.
4. What is an identifier?
5. Define a constant.
6. What is a token?
7. Define a data type.
8. What is a compiler?
9. What is an interpreter?
10. What is an operator?
11. What is an expression?
12. What is type conversion?
13. What is operator precedence?
14. What is a format specifier?
15. What is an escape sequence?

5 Marks

1. Explain the structure of a C program with a suitable example.
2. Explain the fundamental data types available in C.
3. Explain different types of operators in C with examples.
4. Explain different logical operators in C with examples.
5. Explain keywords, identifiers, constants and variables with examples.
6. Differentiate between compiler and interpreter.

10 Marks

1. Explain the structure of a C program with a suitable example.
2. Explain the characteristics/features of C language.
3. Explain different operators in C with suitable examples.
4. Explain tokens in C and explain their different types.